An Introduction to Lean4

Software Verification

MATH230

School of Mathematics and Statistics
University of Canterbury

Outline

Software Specifications

Lean is first and foremost a programming language. Which means we can write the programs we would normally write; as well as programs that are being used as proofs.

In fact, we can write programs as we normally would and then, all within Lean, prove that they meet the specifications we expect.

Previously we proved theorems about addition and multiplication on the natural numbers. These are just claims about how functions on the data type $\mathbb{N}$ interact. There is no reason why we can't write different programs on different data types and prove properties about those programs.

First order logic (aka dependent types) can be used to write specifications for software!

This is not unit testing with cases.

List

Lists can be created in two ways; stating the emtpy list, or cons-ing an element to the beginning of another list.

```
inductive List (α : Type) where
  | null : List α
  | cons : (a : α) →  (as : List α) → List α
```

This type has a very similar structure to the type of natural numbers. Writing functions out of this type will also be done by pattern matching; accounting for the null case, and the cons case.

append ++

Append is a function which takes two lists and concatenates them into a single list; the first onto the beginning of the second. Here is a recursive definition of append using pattern matching on the first argument:

```
def append {α : Type} (as bs : List α) : List α :=
  match as with
  | null       => bs
  | cons a as => cons a (append as bs)
```

Specifications for Append

This program should have each of the following properties:

```
def null_append {α : Type} :
  ∀ xs : List α, (null ++ xs) = xs

def cons_append {α : Type} :
  ∀ x : α, ∀ xs ys : List α,
    ((cons x xs) ++ ys) = (cons x (xs ++ ys))

theorem append_null {α : Type} :
  ∀ xs : List α, (xs ++ null) = xs

theorem append_cons {α : Type} :
  ∀ xs : List α, (xs ++ null) = xs

theorem append_assoc {α : Type} :
  ∀ xs ys zs : List α, (xs ++ ys) ++ zs = xs ++ (ys ++ zs)
```

Definitionally Equal

These are definitionally equal!

```
def null_append {α : Type} :
  ∀ xs : List α, (null ++ xs) = xs

def cons_append {α : Type} :
  ∀ x : α, ∀ xs ys : List α,
    ((cons x xs) ++ ys) = (cons x (xs ++ ys))
```

Pen-and-Paper Proof

What would a proof of a statement like this even look like?

```
theorem append_null {α : Type} :
  ∀ xs : List α, (xs ++ null) = xs
```

Induction on Lists

This suggests that we can do induction on lists; which shouldn't be too suprising given that the type List α is very similar to $\mathbb{N}$ the type of natural numbers.

```
theorem append_null {α : Type} :
  ∀ xs : List α, (xs ++ null) = xs :=
    by
      intro as
      induction as with
      | null          => _
      | cons k ks ih => _
```

Pen-and-Paper Proof

```
theorem append_cons {α : Type} :
  ∀ xs : List α, (xs ++ null) = xs
```

Pen-and-Paper Proof

```
theorem append_assoc {α : Type} :
  ∀ xs ys zs : List α, (xs ++ ys) ++ zs = xs ++ (ys ++ zs)
```

Pen-and-Paper Proof

```
def length {α : Type} : List α → Nat
  | null       => 0
  | cons _ as => 1 + length as

theorem length_append {α : Type} :
  ∀ xs ys : List α, length (xs ++ ys) = length xs + length ys
```

Software Verification

With these examples we begin to see how properties of programs can be expressed in the dependent type system of Lean4. Moreover, we can write programs that inhabit these properties/propositions in order to prove them. In this way we are able to verify programs meet certain specifications *for all possible inputs*.

Example: One can implement merge_sort in Lean4 and also write a specification to say what it means for a list to be sorted. This would be a unary predicate, Sorted, on List. Finally, one could prove the theorem:

```
theorem merge_sorted {α : Type} :
  ∀ xs : List α, Sorted (merge_sort xs)
```

Programming and verification all happen in the same environment.